

RESUMEN – MÉTODOS DE SCANNER

Método	Qué hace
<code>nextLine()</code>	Lee una línea completa como String
<code>next()</code>	Lee la siguiente "palabra" (hasta espacio)
<code>nextInt()</code>	Lee el siguiente entero
<code>nextDouble()</code>	Lee el siguiente decimal
<code>nextBoolean()</code>	Lee true o false
<code>hasNextLine()</code>	¿hay más líneas?
<code>hasNextInt()</code>	¿el siguiente token es un int?
<code>hasNext()</code>	¿hay más tokens?
<code>close()</code>	Cierra el Scanner

FUENTES DE DATOS PARA SCANNER

Fuente	Constructor
Teclado	<code>new Scanner(System.in)</code>
Archivo	<code>new Scanner(new File("ruta"))</code>
String en memoria	<code>new Scanner("texto aqui")</code>

TOSTRING() – REPRESENTAR OBJETOS

`System.out.println(obj)` llama internamente a `obj.toString()`. Si no lo sobrescribes, Java imprime la dirección de memoria (`Persona@1a2b3c`).

```
class Persona {
    String nombre; int edad;
    @Override
    public String toString() {
        return "Nombre: " + nombre
            + " | Edad: " + edad;
    }
}

// Usa:
Persona p = new Persona();
p.nombre = "Gabriel"; p.edad = 20;
System.out.println(p); // Nombre: Gabriel | Edad: 20
```

X ERRORES COMUNES

- Usar `nextInt()` seguido de `nextLine()` → el `nextLine()` lee el `\n` que dejó el int y devuelve cadena vacía. Añadir un `sc.nextLine()` extra como "consumidor" del salto.
- No validar con `hasNextInt()` antes de `nextInt()` → `InputMismatchException` si el dato no es numérico
- No cerrar `Scanner` → `resource leak` (usar `try-with-resources` o `sc.close()`)
- Crear `Scanner` sobre `File` lanza `FileNotFoundException` (comprobada) → obligatorio `try/catch` o `throws`

EJEMPLO COMPLETO – SCANNER + FILEWRITER

Patrón integrador: pedir datos al usuario con `Scanner(System.in)` y guardarlos en un archivo con `FileWriter`.

```
import java.io.FileWriter;
import java.io.IOException;
import java.util.Scanner;

public class GuardarDatos {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        try (FileWriter fw =
            new FileWriter("datos.txt")) {

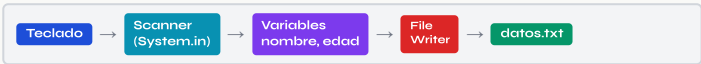
            System.out.print("Nombre: ");
            String nombre = sc.nextLine();

            System.out.print("Edad: ");
            int edad = sc.nextInt();

            fw.write("Nombre: " + nombre + "\n");
            fw.write("Edad: " + edad);
            System.out.println("Datos guardados");

        } catch (IOException e) {
            System.out.println("Error");
        }
        sc.close();
    }
}
```

FLUJO DEL EJEMPLO INTEGRADOR



⚠ PROBLEMA NEXTINT() + NEXTLINE()

Después de `nextInt()`, el buffer retiene el `\n` del Enter. El siguiente `nextLine()` lo consume y devuelve `" "` en vez del texto esperado.

```
Scanner sc = new Scanner(System.in);
int edad = sc.nextInt();
sc.nextLine(); // ← consume el \n residual
String nombre = sc.nextLine(); // ← ahora funciona
```

📁 PATRONES DE USO FRECUENTES

REGISTRO DE MÚLTIPLES USUARIOS HASTA "FIN"

```
Scanner sc = new Scanner(System.in);
FileWriter fw = new FileWriter("usuarios.txt", true);
String linea;
while (!(linea = sc.nextLine()).equals("fin")) {
    fw.write(linea + "\n");
}
fw.close(); sc.close();
```

SUMA DE NÚMEROS LEÍDOS DESDE ARCHIVO

```
Scanner sc = new Scanner(new File("nums.txt"));
int suma = 0;
while (sc.hasNextInt()) suma += sc.nextInt();
System.out.println("Suma: " + suma);
```

DAM · PROGRAMACIÓN · FLUJOS (I/O)

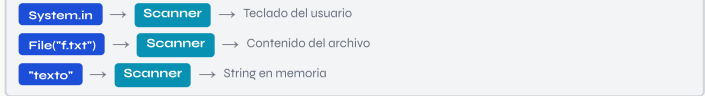
Interacción Estándar y Scanner

BLOQUE 3 · CHULETARIO · TRÍPTICO A4

¿QUÉ ES SCANNER?

La clase `Scanner` (`java.util`) permite leer datos de distintas fuentes — teclado, archivos o Strings — con métodos adaptados a cada tipo de dato. Valida antes de leer con métodos `hasNext*()`.

FUENTES DE SCANNER



CONTENIDOS DE ESTE TRÍPTICO

- Interior izq.** — Scanner desde archivo y comprobación de datos
- Interior cent.** — Scanner desde teclado, `toString()`, buenas prácticas
- Interior der.** — Tabla comparativa de métodos, resumen del bloque
- Solapa** — Ejemplo integrador Scanner + `FileWriter`, patrón `nextInt/nextLine`
- Contraportada** — Todos los métodos, fuentes, `toString`, errores comunes

BLOQUE 3 EN CONTEXTO

Bloque	Clases clave	Tipo de dato
1 – Bytes	<code>FileInputStream/OutputStream</code>	binario
2 – Texto	<code>FileReader/Writer + Buffered*</code>	caracteres
3 – Scanner	<code>Scanner + File/System.in</code>	tipos Java

CARA A → Exterior · CARA B → Interior (teoría y referencia)
Imprimir: A4 horizontal · doble cara · voltear por lado largo

🔥 SCANNER – LEER DESDE UN ARCHIVO

Scanner no es solo para teclado. Acepta un `File` como fuente y permite leer archivos de texto con los mismos métodos cómodos.

EJEMPLO COMPLETO

```
import java.io.File;
import java.io.IOException;
import java.util.Scanner;

public class LeerArchivoScanner {
    public static void main(String[] args) {
        try {
            Scanner sc =
                new Scanner(new File("archivo.txt"));
            while (sc.hasNextLine()) {
                String linea = sc.nextLine();
                System.out.println(linea);
            }
            sc.close();
        } catch (IOException e) {
            System.out.println("Error al leer");
        }
    }
}
```

Código	Qué hace
<code>new Scanner(new File("f"))</code>	Abre el archivo como fuente del Scanner
<code>sc.hasNextLine()</code>	Comprueba si hay más líneas antes de leer
<code>sc.nextLine()</code>	Lee la línea completa como String
<code>sc.close()</code>	Cierra el Scanner y el archivo
<code>catch IOException</code>	Obligatorio – <code>FileNotFoundException</code> es checked

✅ COMPROBACIÓN DE DATOS – VALIDAR ANTES DE LEER

Los métodos `hasNext*()` permiten comprobar el tipo del siguiente token **sin consumirlo**. Evitan excepciones si el dato no es del tipo esperado.

LEER SOLO ENTEROS VÁLIDOS

```
while (sc.hasNextInt()) {
    int num = sc.nextInt();
    System.out.println(num);
}

// Si el siguiente token no es int, el bucle termina
// sin lanzar InputMismatchException
```

FILTRAR SOLO POSITIVOS

```
while (sc.hasNextInt()) {
    int n = sc.nextInt();
    if (n > 0) System.out.println(n);
}
```

TABLA HASNEXT* COMPLETA

Método	Valida
<code>hasNextLine()</code>	¿hay más líneas?
<code>hasNext()</code>	¿hay más tokens?
<code>hasNextInt()</code>	¿siguiente token es int?
<code>hasNextDouble()</code>	¿siguiente token es double?
<code>hasNextBoolean()</code>	¿siguiente token es boolean?

Regla: usa siempre `hasNext*()` antes del `next*()` correspondiente cuando leas archivos o entradas externas.

🖨️ SCANNER – LEER DESDE TECLADO

```
import java.util.Scanner;

Scanner sc = new Scanner(System.in);

System.out.print("Introduce tu nombre: ");
String nombre = sc.nextLine(); // línea completa

System.out.print("Introduce tu edad: ");
int edad = sc.nextInt();      // solo el número

System.out.println("Hola " + nombre + ", tienes " + edad);
sc.close();
```

NEXT() VS NEXTLINE()

Método	Lee hasta	Ejemplo entrada	Resultado
<code>next()</code>	Espacio / tab / \n	"Hola mundo"	"Hola"
<code>nextLine()</code>	Salto de línea \n	"Hola mundo"	"Hola mundo"

🗑️ toString() – REPRESENTAR OBJETOS

Cuando pasas un objeto a `System.out.println()`, Java llama automáticamente a `toString()`. Sobrescribe este método para controlar cómo se muestra el objeto.

```
class Persona {
    String nombre;
    public Persona(String nombre) {
        this.nombre = nombre;
    }
    @Override
    public String toString() {
        return "Nombre: " + nombre;
    }
}

Persona p = new Persona("Gabriel");
System.out.println(p);
// → Nombre: Gabriel
// Sin @Override → Persona@4a5a6b7c
```

Sin @Override	Con @Override
<code>Persona@1a2b3c</code> dirección de memoria	<code>Nombre: Gabriel</code> texto legible

✅ BUENAS PRÁCTICAS CON SCANNER

<code>sc.close()</code> siempre o usar try-with-resources	<code>hasNext*()</code> antes de leer evita InputMismatchException
<code>nextLine()</code> para textos captura espacios y línea completa	Limpiar buffer tras <code>nextInt()</code> añadir <code>sc.nextLine()</code> extra
Validar entradas externas archivos y teclado pueden tener errores	<code>@Override toString()</code> en toda clase con datos relevantes

🔥 SCANNER VS BUFFEREDREADER – ¿CUÁNDO USAR CADA UNO?

	Scanner	BufferedReader
Lee por tipos	✓ <code>nextInt()</code> , <code>nextDouble()</code> ...	✗ solo String
Valida antes	✓ <code>hasNext*()</code>	✗ no
Velocidad	Más lento	Más rápido
Leer línea	<code>nextLine()</code> → String	<code>readLine()</code> → String
Fin de archivo	<code>hasNextLine()</code> = false	<code>readLine()</code> = null
Ideal para	Teclado / entradas mixtas	Archivos de texto grandes

📦 PATRONES DE USO – COMPARATIVA DE FUENTES

SCANNER DESDE TECLADO

```
Scanner sc = new Scanner(System.in);
String texto = sc.nextLine();
sc.close();
```

SCANNER DESDE ARCHIVO

```
Scanner sc = new Scanner(new File("f.txt"));
while (sc.hasNextLine())
    System.out.println(sc.nextLine());
sc.close();
```

SCANNER DESDE STRING (ÚTIL EN TESTS)

```
Scanner sc = new Scanner("10 20 30");
while (sc.hasNextInt())
    System.out.println(sc.nextInt());
sc.close();
```

🗑️ TABLA RESUMEN DEL BLOQUE

Operación	Método	Devuelve
Leer línea completa	<code>nextLine()</code>	String
Leer palabra	<code>next()</code>	String
Leer entero	<code>nextInt()</code>	int
Leer decimal	<code>nextDouble()</code>	double
Leer booleano	<code>nextBoolean()</code>	boolean
¿más líneas?	<code>hasNextLine()</code>	boolean
¿siguiente es int?	<code>hasNextInt()</code>	boolean
Representar objeto	<code>toString()</code>	String
Cerrar Scanner	<code>close()</code>	void

Regla de oro: usa `Scanner(System.in)` para leer del usuario · `Scanner(new File())` para archivos · valida siempre con `hasNext*()` · sobrescribe `toString()` en tus clases.